Release Plan – VeriFi – Revisions: 2 – Revision Date: 01/July

High Level Goals:

1. Data Ingestion
2. Vector Store
    a. Implement an algorithm that best replicated cosine similarity to match parsed chunks to user inquiry
        i. Limitations: Will most likely have to implement it myself rather than use a library due to the vector database being custom made – will take up a lot of time
    b. Properly receive the parsed info from the data ingestion role
    c. After ranking chunks, feed the information back into the backend API
    d. Properly store information and create a function to search for relevant information depending on scores
    e. Build a data structure that can efficiently store the parsed information and retrieve it within O(N) time
        i. Would have to extensively research a data structure that would allow for universal access across the program

3. API Backend Bridge
4. Frontend
5. Prompt Engineering, Integration & Quality Assurance
    a. Design and refine prompts for Retrieval-Augmented Generation (RAG).
    b. Ensure responses remain grounded in retrieved Fidelity documents.
    c. Develop evaluation criteria and QA test cases for system validation.
    d. Validate prompt quality and response accuracy throughout integration.

User Stories:

- **Sprint 1**
    - {priority} User story 1.1 [As a Fidelity user, I want to be able to type a query into a web chatbox and receive a generic text acknowledgment, so that I know the system is successfully receiving my input.]
    - {priority} User story 1.2 [As a Fidelity user, I want the chatbox to provide a clickable link to the source document alongside my answer, so that I can verify the information myself.]
    - {priority} User story 1.3 [As a system administrator, I want to be able to upload raw financial policy text files into the system, so that the custom semantic vector database is populated with the necessary knowledge to process user queries.]
    - {priority} User Story 1.4 [As a front-end developer, I want a running /chat endpoint with a defined request/response contract, so that I can begin

building the UI against mocked data without waiting on the full pipeline.]

- **Sprint 2**
  - {priority} User story 2.1 [As a programmer maintaining the project, I want a clean file structure and commentated function so that it is easier to update and improve the functionality / pipeline.]
  - {priority} User story 2.2 [As a user of the program, I want to see a website design that does not overcomplicate things for me so that I can ask my questions and receive my answer without any hurdles]
  - {priority} User story 2.3 [As a Fidelity user, I want the AI to answer my financial policy questions using only the information retrieved from official company documents so that I receive accurate, verifiable and trustworthy responses]
  - {priority} User story 2.4 [As a developer, I want a testing plan for VeriFi so that the system can be evaluated consistently as new features are added.]
  - {priority} User Story 2.5 [As a front-end developer, I want a running /chat endpoint with a defined request/response contract, so that I can begin building the UI against mocked data without waiting on the full pipeline.]

- **Sprint 3**
  - {priority} User story 3.1 [As a Fidelity user, I want to be able to view my chat history.] (2)
  - {priority} User story 3.2 [As a Database Engineer, I want to make sure that cosine-search is accessible to the backend] (2)
  - {priority} User story 3.3 [As a Backend Engineer, I want to make sure requests are properly received and sent to the frontend] (3)
  - {priority} User story 3.4 [As a Fidelity user, I want responses to be generated using retrieved documents and include document citations so that I can trust the information provided.] (3)
  - {priority} User story 3.5 [As a developer, I want to validate the complete VeriFi pipeline after integration so that the frontend, backend, vector database, retrieval system, and LLM communicate correctly and users receive accurate, cited responses] (3)

- **Sprint 4**
  - {priority} User story 4.1 [As a Fidelity user, I want only relevant information to be displayed to me.] (3)
  - {priority} User story 4.2 [As a Systems Administrator, I want the user to only be able to access specific policy information that is requested] (5)
  - {priority} User story 4.3 [As a Fidelity User, I do not want long wait times between questions] (5)
  - {priority} User story 4.4 [As a Fidelity user, I want clear, concise, and accurate AI responses so that financial policy information is easy to understand] (3)
  - {priority} User story 4.5 [As a developer, I want to perform end-to-end testing and resolve remaining bugs so that VeriFi is reliable before the final release] (5)

Recall that a user story should take the form, "As a {user role}, I want {goal} [so that {reason}]". User stories should meet the "INVEST" criteria (independent, negotiable, valuable, estimatable, sized appropriately, and testable).

It is a good idea to identify each user story by a unique label that allows the user story to be referenced across different tools and documents.

Sanity Check:

We have team members who are versed in full-stack development and C++ development. We strategically divided the roles in such a way that we each work on an area of the project which we feel most comfortable, avoiding unnecessary learning curves and using time ineffectively. Based on the user stories, where the main point of them is that the program should allow users to receive accurate, documented sources of information back for their inquiries, the team has full capacity at creating a site for individuals to at the minimum request policy information and receive an answer whether structured or brute-forced. The Prompting and QA role complements this goal by ensuring the AI only generates responses grounded in retrieved Fidelity documents and by validating the accuracy, formatting, and citation quality of generated answers throughout development.

The work distribution across the sprints is reasonable, especially considering the way we divided the roles it allows us to work independently outside of group meetings. We ensured to allocate time for not just spikes and group meetings, but also dedicated small gaps to re-group and update each other on our progression through the project and any updated expectations. In terms of holidays or midterms, we have already worked out a method to ensure we can enjoy time off without hindering progression or our teammates.

Product Backlog:

A listing of all high-level goals and user stories that were discussed in the release planning meeting, but which did not make it into the release at this point. User story priorities may change in the course of the project and therefore the PO may decide to downgrade some user stories currently in the release plan and promote some user stories currently in the backlog. The release plan and product backlog should be revisited and updated after each sprint.

The product backlog remaining at the end of the last sprint can serve as the starting point for a subsequent release.

INFO BELOW:

- **High level goals:** A description of the top-level goals for the release. Examples include, "Have all controller capabilities implemented," "Be able to create levels using a level
  for a game: "Be able to play one complete level (but with limitations xx, yy, & zz)," design tool;" or for the Osric system: “Be able to handle service requests for new and existing customers with access to requests by managers and technicians.” These high level goals may map to a single user story, but more typically will map to multiple user stories.

- **User stories defining the scope of the release:** A listing of all the user stories that are needed to implement the high-level goals. Each user story must have a level of effort estimate in story points. Each user story must be sized to fit within a single sprint. Each user story must be assigned to one of the sprints within the development period (usually 4 two-week sprints in a quarter-length course; 3 one-week sprints in a five week summer course).

  Either list the user stories in priority order within each sprint or indicate the priority of each user story explicitly.

  Recall that a user story should take the form, "As a {user role}, Iwant {goal} [so that {reason}]". User stories should meet the "INVEST" criteria (independent, negotiable, valuable, estimatable, sized appropriately, and testable).

  It is a good idea to identify each user story by a unique label that allows the user story to be referenced across different tools and documents.

Last modified: 2026-07-01